AI Engineering and Claude Architect · Part II, Retrieval

RAG, Lesson 5

The RAG Family: Graph, SQL, Agentic, Multimodal, Self-RAG

"Which themes run through all 40,000 customer complaints?" Search returned 20 complaints that mention themes, and the assistant summarized those 20 as if they were the whole set. The pipeline worked exactly as built. It was the wrong shape for the question.

Module	AI05 of 13
Lesson	5 of 7
Prerequisites	AI05 L1–L4, AI04 L3
Exam relevance	Out of scope by name. Agentic retrieval relates to TS 2.5 and D1, and tool-based retrieval to TS 2.1
Reading time	~40 minutes

Contents

1. The RAG Family

Lesson 5 of 7 · RAG

The RAG Family

Key idea

Each RAG variant exists because a certain kind of question defeats top-k passage search. Whole-corpus questions need precomputed summaries, numeric questions need SQL, and multi-step questions need an agent. **So choose from the questions you actually receive, and add a variant only when your evaluation shows that plain hybrid retrieval fails for that kind of question.**

> *Before: the standard pipeline and hybrid search. Teaches: nine retrieval shapes, what each one fixes and costs, and how to validate model-written SQL. Used by: AI05 L7, AI06 L3, AI06 L6.*

The hook: top-k can't see the whole

Top-k search answers one question: which passages look most like this query? That's the wrong operation for several common requests.

Question	Why top-k fails	What it needs
"What themes run through all complaints?"	No passage contains the whole	Precomputed summaries
"How many refunds last quarter?"	Passages can't sum	SQL
"Who manages the team that owns the billing service?"	Facts in different documents link together	Several hops
"What does this architecture diagram show?"	The meaning is in the image	Multimodal
"What's my order status?"	It isn't in a document	An API

The GraphRAG paper puts the first case directly: "RAG fails on global questions directed at an entire text corpus."

Intuition: a reference desk

A good reference desk doesn't only have a card catalogue. It has an **encyclopedia's overview articles** for broad questions, a **statistics office** for counts, a **telephone** for what's changing today, a **picture archive**, and an **experienced researcher** who follows a trail through several sources. The catalogue remains the default. The librarian's skill lies in knowing which to use.

The nine shapes

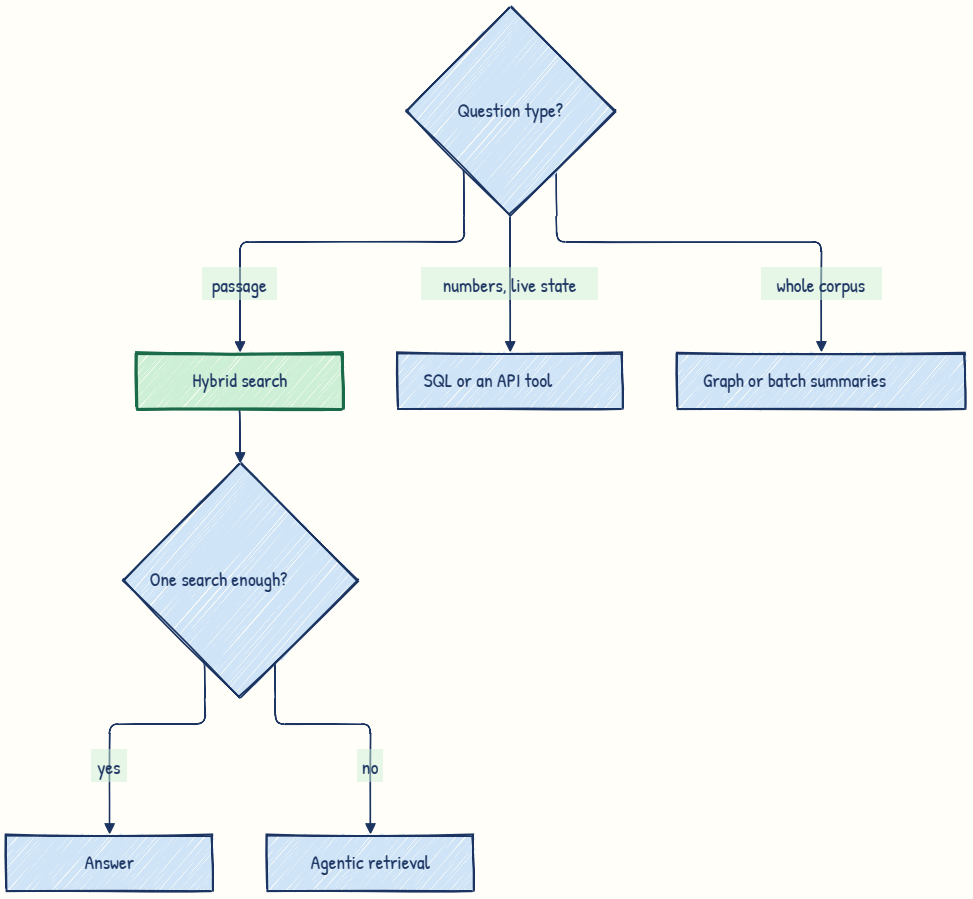


Figure 1. Choosing a retrieval shape from the question. Start with hybrid search, and branch only for question types that it demonstrably can't serve.

Shape	Fixes	New cost	Reach for it when
Vector	Meaning-based matching	Weak on exact terms	Rarely alone
Hybrid (AI04 L3)	Both meaning and exact terms	Two indexes	**The default**
Vectorless	Pipeline overhead and chunking	Slower, and more tokens per question	Code, structured documents, modest size
Graph	Whole-corpus and multi-hop questions	Expensive indexing, with extraction errors	Those questions are common and valuable
SQL	Aggregations, exact values	Wrong queries look right	Facts live in tables
API or tool	Live data, actions	Each one to build and secure	Facts live in a service
Agentic	Multi-step questions	Latency, cost, loops	Steps can't be planned in advance
Multimodal	Charts, scans, diagrams	Storage, slower processing	Meaning lives in images
Self-RAG, or adaptive	Retrieving when unnecessary, or poorly	Complexity, and more calls	Quality is failing and evals show why

Vectorless: search without embeddings

Not all retrieval needs vectors. Anthropic's context engineering post describes Claude Code using this approach: `CLAUDE.md` loaded up front, with `glob` and `grep` for navigating on demand. That avoids stale indexes and complex syntax trees.

Works when	Fails when
Text has strong exact terms, such as code	Vocabulary differs from the query
The corpus has a navigable structure	The corpus is huge, and unstructured
Answers can take seconds	Interactive latency matters

A related approach gives a model a document's table of contents, and lets it pick sections to read. It's costlier per question, but it needs no chunking. Its best case is a few long, well-structured documents, such as filings or manuals. That description comes from general knowledge, not from a paper read for this course.

Graph RAG

GraphRAG (Edge and colleagues, 2024) precomputes what top-k can't find at query time.

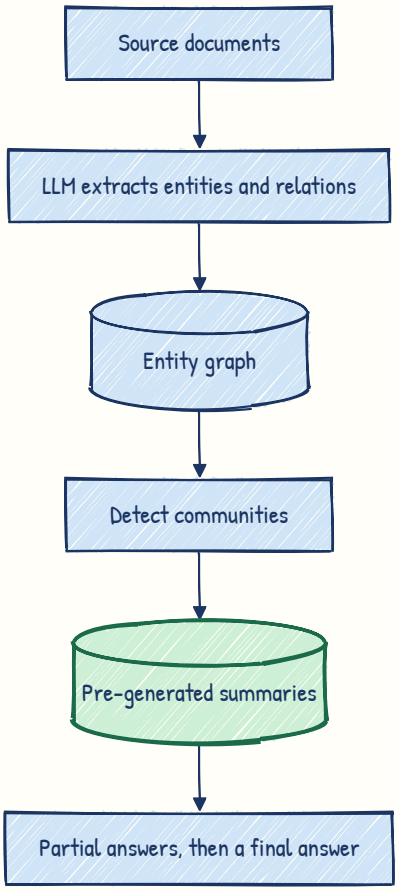


Figure 2. GraphRAG, as the paper describes it. An LLM extracts an entity graph, communities are found and summarized in advance, and answers are built from those summaries.

Note the cost. Every document goes through an LLM before anyone asks a question, and updates may mean recomputing communities. Extraction errors also become wrong edges that look authoritative.

Often a much lighter design is enough:

Your actual need	Lighter option
"Summarize all complaints"	A batch job: map-reduce over the corpus, for one known question
"What are the themes?"	Periodic clustering, with a summary per cluster
"Who manages X?"	Your HR or service directory, through a tool

SQL RAG, and what Uber found

Uber's QueryGPT write-up is one of the more detailed public accounts of building this.

The problem. About 1.2 million interactive queries a month, and they estimated a query took about 10 minutes to author by hand, or about 3 with QueryGPT.

What failed. Their first version used k-NN over schemas and sample queries, retrieving 3 tables and 7 SQL samples. Accuracy declined as they added tables. Some schemas had over 200 columns, at about 40,000–60,000 tokens.

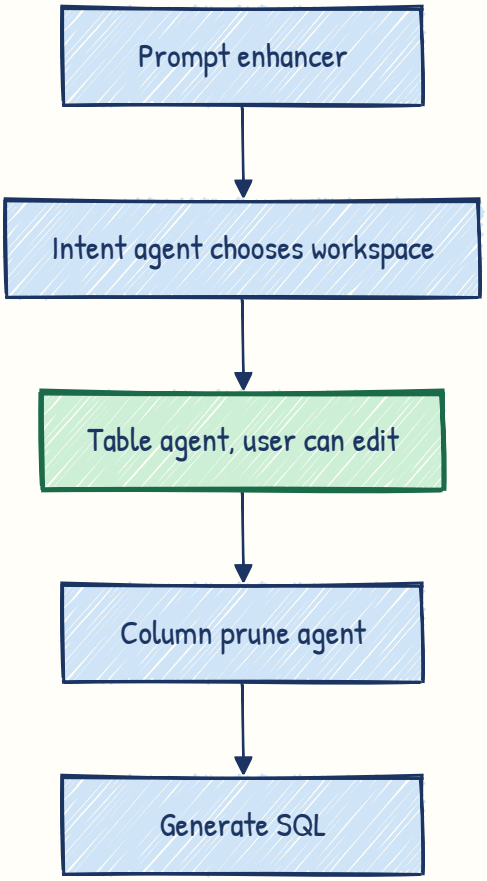


Figure 3. Uber's published design, simplified. The problem is narrowed in stages by small agents before any SQL is written, and the user can correct the chosen tables.

Their finding	Lesson
Plain vector search didn't scale with tables	Domain narrowing beats similarity alone
Curated "workspaces" narrowed the search	Curation is a retrieval technique
LLMs worked well as narrow classifiers	Small focused agents beat one large prompt
Users could correct the chosen tables	Human confirmation early is cheap
Hallucinated tables and columns stayed unsolved, with a validation agent planned	Validate in code

Uber reports about 300 daily users, and about 78% saying that it reduced their time. **It doesn't publish an accuracy figure, so don't quote one.**

Validate before running

Model-written SQL needs a guard. This is from `code/ai05-rag/.../SqlGuard.java`, with real output, condensed to one line per query:

```
if (!lower.startsWith("select") && !lower.startsWith("with")) {
    problems.add("only SELECT is allowed");
}
if (WRITES.matcher(lower).find()) {
    problems.add("contains a write keyword");
}
Matcher tables = TABLES.matcher(lower);
while (tables.find()) {
    if (!schema.containsKey(tables.group(1))) {
        problems.add("unknown table: " + tables.group(1));
    }
}
```

```
SELECT region, SUM(total) FROM orders JOIN customers ...   -> []
SELECT * FROM customer_orders_v2                           -> [unknown table: customer_orders_v2]
DELETE FROM orders                 -> [only SELECT is allowed, contains a write keyword]
SELECT 1; DROP TABLE orders        -> [contains a write keyword, multiple statements]
```

> *This guard is a teaching sketch, and not a security boundary*
>
> It would reject a harmless `WHERE action = 'delete'`, and regex can't understand SQL. **Real protection lives in the database: a read-only role, row-level security, query timeouts and row limits.** Use a real SQL parser too. And a query that's valid can still be wrong, by joining the wrong tables, say. So show users the SQL, and evaluate on known-answer questions.

Feed validation errors back so the model can correct itself. That's the retry-with-feedback pattern from AI03 L3, and TS 4.4.

Agentic RAG

Here, the model decides what to search for, reads the result, then decides what to do next.

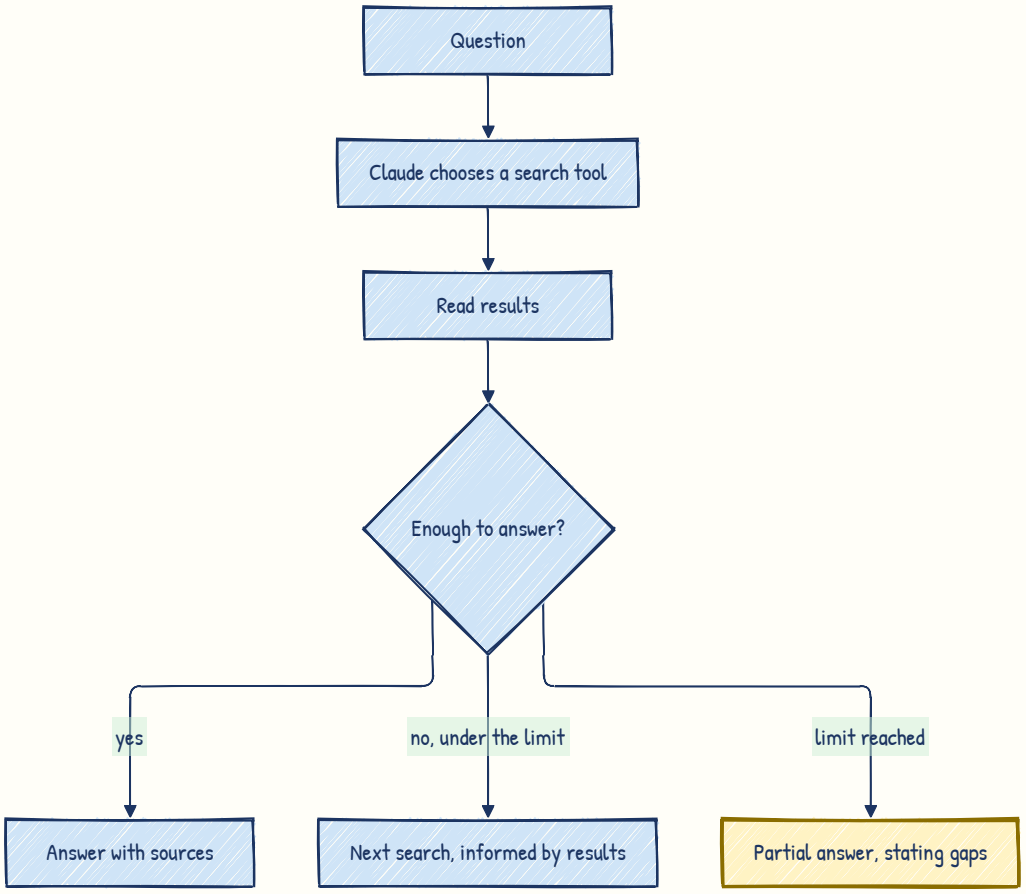


Figure 4. Agentic retrieval as a sequence of decisions. Each search result informs the next choice, and a step limit stops runaway loops.

LinkedIn's retrieval step is in this spirit: an agent chooses which services to call. Agents are good at following a trail. **They cost much more, though, so route only suitable questions to them.** AI06 L3 builds this loop properly, including distinct tool descriptions (TS 2.1) and stopping conditions (TS 1.1).

Multimodal RAG

Approach	How	Fits
Describe, then index text	A vision model writes descriptions, which get embedded	Simple, uses existing text search
Multimodal embeddings	Embed the image directly	Visual similarity. Check the current model guide
Hybrid	Find text with search, then send the original page to Claude	Scanned forms and charts

A practical point from the Citations docs: PDF citations are supported, but they can't point at image content, and scanned PDFs without extractable text can't be cited. So a chart's values won't get a citation pointing at that chart. If auditing matters, keep the page number in your metadata.

Self-RAG and adaptive retrieval

Self-RAG (Asai and colleagues, 2023) trains a model to "adaptively retrieve passages on-demand" and to critique its own output with special reflection tokens. **It requires a specially trained model, so you can't directly apply that method to a hosted model.**

The practical descendant is a set of checks around retrieval:

Check	Action
Does this question need retrieval at all?	Skip it for chit-chat or simple follow-ups
Are the retrieved chunks relevant?	Grade them, and search again if they're weak
Is the top score very low?	Say "I don't know" without asking the model
Is the answer supported?	Check it, and regenerate or flag it

Each check adds latency and cost. Often the cheapest is the score threshold, which needs no model call.

How Anthropic and the field solved it

Need	Approach	Source
Coding context	Just-in-time `glob` and `grep`, with `CLAUDE.md` up front	Anthropic's context engineering post
Better single-shot search	Contextual Retrieval	AI05 L3
Whole-corpus questions	Community summaries	Microsoft Research's GraphRAG paper
Text to SQL	Workspaces and narrow agents	Uber
Service retrieval	Agent-selected service calls	LinkedIn
Adaptive retrieval	Reflection tokens	Self-RAG

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: one retriever for everything. Whole-corpus and counting questions get confident, wrong answers.

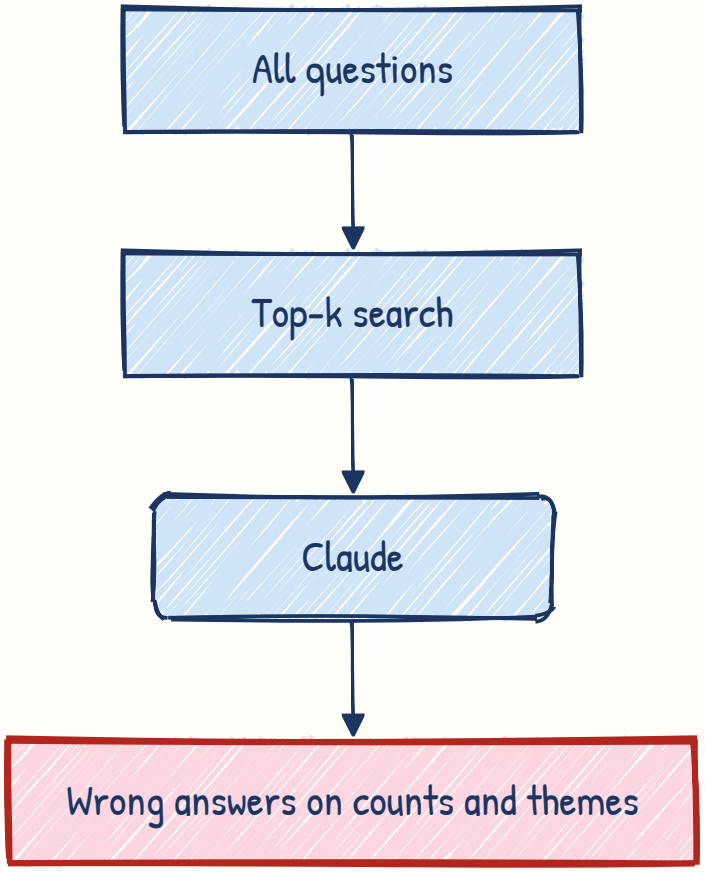


Figure 5. Baseline. Every question goes to one top-k search, whether it suits that or not.

Improved: an agent with every tool. It's flexible. New problem: **simple lookups now take many steps, and cost and latency vary widely.**

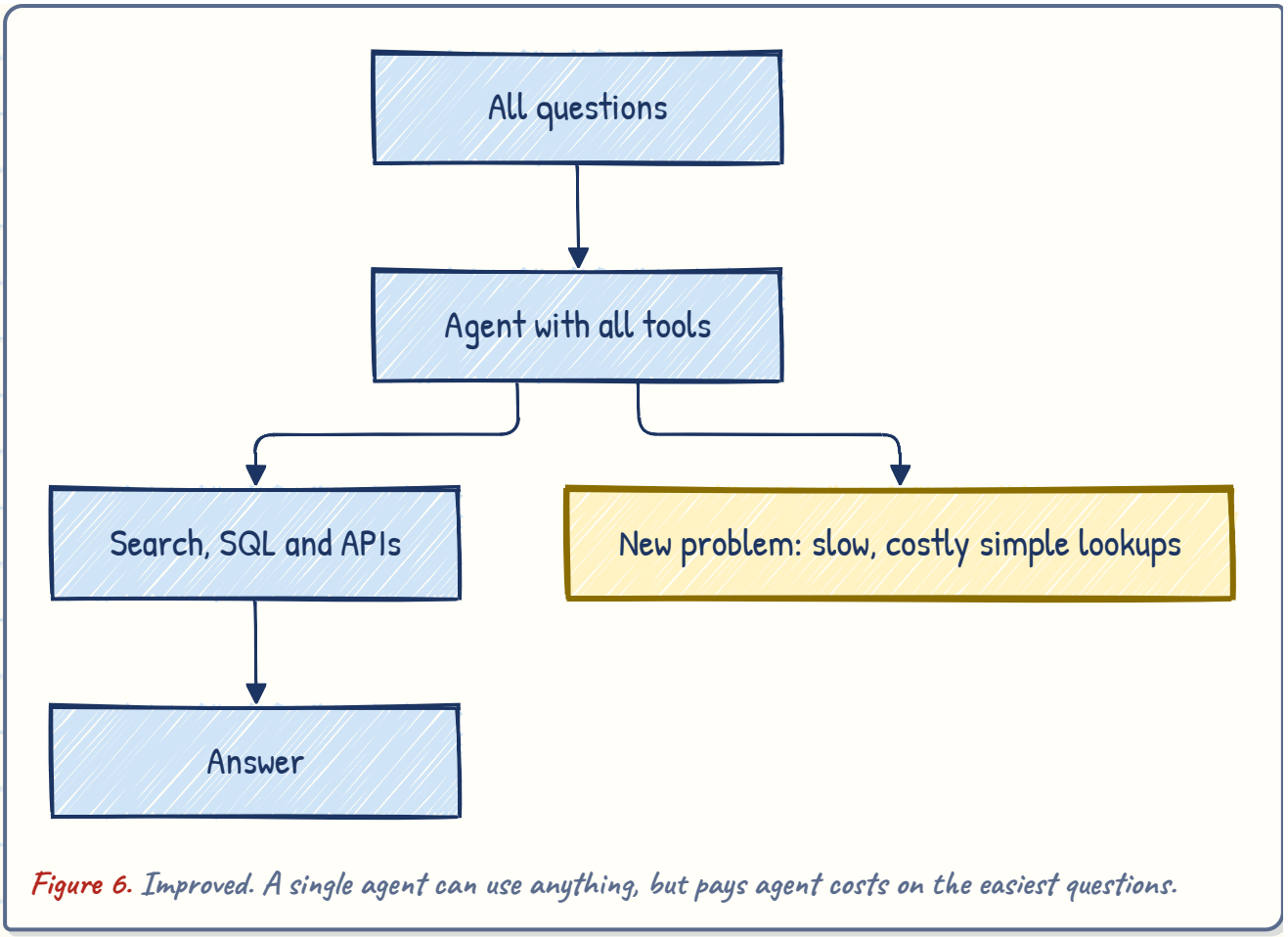


Figure 6. Improved. A single agent can use anything, but pays agent costs on the easiest questions.

Production: cheapest route first.

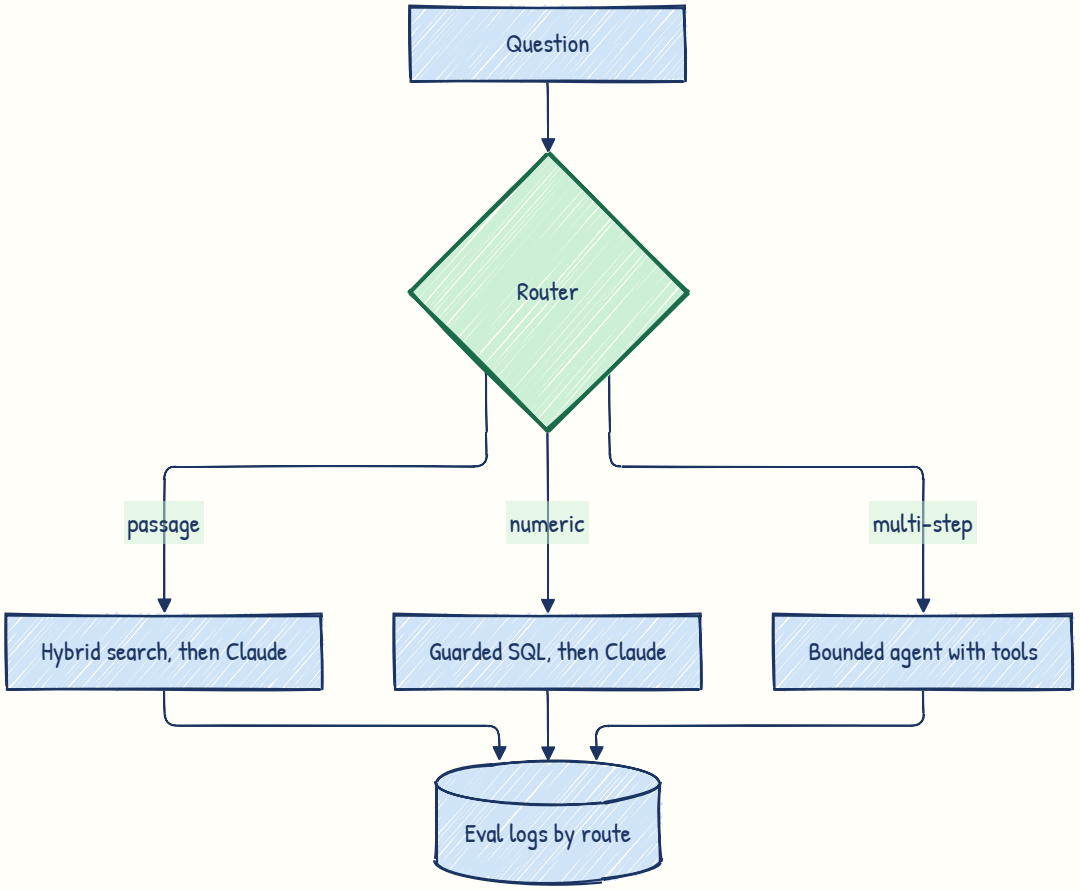


Figure 7. Production. A router sends each question to the cheapest shape that can serve it. Only questions that need several steps go to the agent.

Component	Why it exists	How it communicates
Router	Keeps easy questions cheap	Rules, then a small model
Hybrid path	Most questions	Search API
Guarded SQL	Aggregations	Read-only connection, schema guard
Bounded agent	Multi-step questions	Tool loop with step and budget limits
Eval logs	Tells you whether each route earns its cost	Async

Failures

Failure	Containment
SQL runs but answers the wrong question	Show the SQL, and test against known answers
Agent loops	Step and budget limits, then a partial answer
Router sends counting questions to search	Evaluate by route
Graph holds wrong extracted edges	Sample and review them, and cite the source text
Expensive query locks a database	Read replica, timeouts, row limits

Trade-offs

Decision	Gain	Cost
Adding a graph	Whole-corpus answers	Indexing cost and upkeep
Text to SQL	Self-service analytics	Wrong answers that look right
Agents	Handles the unplanned	Cost, latency and variance
Vectorless	No index to maintain	Slower, and more tokens per question
Router	Efficient routes	One more failure point

When not to add a variant: when you haven't yet measured plain hybrid search on your own questions.

Common mistakes

1. **Choosing an architecture from a list.** Start from your questions.
2. **Building a graph for one recurring question.** A batch job may do.
3. **Trusting a guard built on regex.** Use database permissions.
4. **Assuming valid SQL is correct SQL.** It can answer the wrong question.
5. **Using an agent for every question.** Route the simple ones away.
6. **Calling a grade-and-retry loop "Self-RAG".** It's a useful pattern, but a different method.
7. **Quoting an accuracy number for QueryGPT.** Uber didn't publish one.

What to say in an interview

"I'd classify real questions first. Passages go to hybrid search, numbers to read-only SQL with validation, and only multi-step questions go to a bounded agent. Uber's write-up shows that plain vector search over schemas lost accuracy as tables grew, and narrowing by workspace and letting users correct tables helped. Hallucinated columns stayed a problem, so I'd validate in code and rely on database permissions."

Certification lens

How this lesson maps to CCAR-F

These variants aren't tested by name. The related skills are: tool descriptions for routing (**TS 2.1**), retry with error feedback (**TS 4.4**), and agent loops and stopping (**TS 1.1**).

These are original practice questions for understanding, not exam simulations.

Q Q1 · Component selection · Medium. For each, choose a shape and give a reason: (a) "Average handle time by region last month." (b) "What themes appear across all 2025 incident reviews?", asked weekly. (c) "Which function throws this error?", in a large codebase. (d) "Why was this customer charged twice?", which requires several systems.

A (a) SQL, since it's an aggregation. (b) A scheduled map-reduce summary or clustering job, since one known question doesn't justify a full graph. (c) Vectorless `grep`, since error strings are exact terms. (d) A bounded agent with tools, since the next step depends on the last result.

Q Q2 · Troubleshooting · Medium. Text-to-SQL accuracy was good on 50 tables and is poor on 2,000, and the model is unchanged. Why, and what's the fix?

A Retrieving schemas by similarity gets harder as tables multiply, and similar names compete. Uber reported this pattern. Narrow by domain first using curated workspaces, prune columns, let users confirm the tables, and validate names in code.

Q Q3 · Security · Hard. A colleague says the regex guard makes text-to-SQL safe. Give two inputs or situations that show it isn't, and state what actually provides safety.

A The guard doesn't enforce row access, so a valid `SELECT salary FROM employees` passes if the table's known. An expensive cross join also passes, and it could overload the database. Safety comes from a read-only role, row-level security, a replica, timeouts and row limits. The guard only catches mistakes early.

Revision - Lesson 5

→ Top-k finds similar passages. It can't count, summarize a whole corpus or see live state.

→ Start with hybrid search, and add shapes for measured failures.

→ Vectorless suits exact terms and structure. Claude Code uses grep and glob.

→ GraphRAG precomputes entity graphs and community summaries. Indexing is expensive.

→ A batch job often answers one recurring whole-corpus question.

→ Uber: k-NN over schemas declined as tables grew. Workspaces and narrow agents helped.

→ Uber left hallucinated columns unsolved, and published no accuracy figure.

→ Safety for SQL lives in database permissions. A validator only catches mistakes.

→ Agents follow trails, at a price. Route only suitable questions.

→ Self-RAG needs a specially trained model. Practical systems use checks around retrieval.

Your notes